

02_m3_m4

M3 - Sessionization & Funnel Keystone

Objective

Reconstruct the two entities GA4 never ships as rows - the **session** and the **funnel** - from the raw event stream, at one row per session (PK `session_key`). These are THE keystones:

`int_ga4__sessionized` and `int_ga4__funnel_steps`. They **fail silently** - a wrong MD5 concat order still produces a valid-looking key; a non-monotonic flag still produces a number; a first-touch attribution leak still returns rows. Nothing errors; the numbers are just quietly wrong, and every mart, metric, and agent diagnosis built on top inherits the corruption. So the rule for M3 is **golden-value tests FIRST**: write the dbt `unit_tests` and the two singular tests, watch them fail on synthetic input, then make them pass. Get the canonical `session_key` (`COUNT(DISTINCT session_key)` everywhere), the `traffic_source` first-touch FALLBACK (prefer session-scoped `event_params.source/medium`, fall back to `traffic_source` only when null), and the `reached_*` max-downstream monotonic flags exactly right.

Files to create

Path	Copy from
<code>models/intermediate/int_ga4__sessionized.sql</code>	DBT_GUIDE.md §4.1 (full SQL); cross-check DATA_MODEL.md §5.5 (grain/PK/columns) and §5.2-5.4 (derived attrs)
<code>models/intermediate/int_ga4__funnel_steps.sql</code>	DBT_GUIDE.md §4.2 (full SQL); cross-check DATA_MODEL.md §5.6 (max-downstream flags)
<code>models/intermediate/int_ga4__schema.yml</code>	DBT_GUIDE.md §4.3 (structural tests: <code>session_key</code> unique+not_null, <code>channel_group</code> accepted_values of the 10, <code>engaged_session / is_new_user</code> not_null, <code>session_revenue</code> range)
<code>models/intermediate/_int__unit_tests.yml</code>	DBT_GUIDE.md §6.5 (the 3 keystone golden-value unit tests - WRITE FIRST)

Path	Copy from
tests/assert_funnel_monotonicity.sql	DBT_GUIDE.md §6.3 (singular: returns offending sessions; 0 rows = pass)
tests/assert_session_conversion_rate_bounds.sql	DBT_GUIDE.md §6.3 (singular: conv-rate stays in plausible bounds)

Note `tests/assert_session_conversion_rate_bounds.sql` references `fct_daily_funnel` (an M4 mart), so it only goes green once M4 lands; author it now (TDD) but expect it red until then. `assert_funnel_monotonicity.sql` references `fct_funnel` (also M4) - the *intermediate-grain* monotonicity is guarded immediately by the `test_reached_flags_are_max_downstream` unit test, which runs against `int_ga4__funnel_steps` on synthetic input.

Both intermediates are materialized **ephemeral** (inlined as CTEs into their mart consumers - no storage, no extra scan). They are plumbing: never exposed to BI or the semantic layer directly. The semantic layer references **only marts**; marts (`fct_sessions` , `fct_funnel` , `fct_daily_funnel`) are built *from* these intermediates. Because they are internal and inlined, you cannot `dbt run --select` them in isolation and see a table - you validate them through `dbt compile` (refs/macros resolve) plus their unit/singular tests. The default is `ephemeral` ; switch a single intermediate to `view` only if several marts consume it and inlining would duplicate the scan.

Two derivation rules in `int_ga4__sessionized` are easy to get wrong and earn explicit golden tests: `is_new_user` is derived from `ga_session_number = 1` (the user's first-ever session), and `engaged_session` from the canonical rule `session_engaged = '1' OR engagement_time_msec ≥ 10000` (use `≥` , no extra clauses). `channel_group` flows from the **resolved** session-scoped source/medium (after the first-touch fallback) through `channel_group_case()` - never from the raw `traffic_source` struct.

Commands to run

```
# activate venv (Windows: .venv\Scripts\activate ; POSIX: source .venv/bin/activate)
.venv\Scripts\activate

# build staging first (M2 dependency must be green), then the intermediates.
# ephemeral models are not built standalone - they compile into consumers, so
# you validate them via `compile` + their unit/singular tests.
dbt deps
dbt compile --select int_ga4__sessionized int_ga4__funnel_steps # macros + refs resolve

# run the keystone unit tests (golden values) - these are the real gate
dbt build --select staging # ensure upstream is green
dbt test --select int_ga4__sessionized int_ga4__funnel_steps # unit + schema tests
```

```
# or by tag once tagged:
dbt build -s tag:keystone
```

On the static sample always reset with `dbt build --full-refresh` (3 months rebuild in seconds); the `is_incremental()` branch is production-only and never fires here.

Tests to run

```
# 1. Golden-value unit tests (synthetic input, <1s, every build) - DBT_GUIDE §6.5
dbt test --select int_ga4__sessionized,test_type:unit
dbt test --select int_ga4__funnel_steps,test_type:unit
#   asserts: u1+session 100 collapses 2 events → 1 session; gclid+cpc ⇒ 'Paid Search';
#             a session whose ONLY event is 'purchase' back-fills ALL upstream reached_* = true;
#             a duplicated purchase counts session_revenue ONCE.

# 2. Structural schema tests - DBT_GUIDE §4.3
dbt test --select int_ga4__sessionized int_ga4__funnel_steps
#   asserts: session_key unique + not_null on BOTH; channel_group in the 10 accepted values.

# 3. Singular invariants - DBT_GUIDE §6.3 (green after M4 marts exist)
dbt test --select assert_funnel_monotonicity assert_session_conversion_rate_bounds
#   monotonicity: 0 rows where reached_purchase > reached_add_payment_info > ... >
#                 reached_view_item
#   conv-bounds: no day with conv_rate >1, <0.0005, or >0.20
```

Success criteria

- `session_key` is **unique** and **not null** on both `int_ga4__sessionized` and `int_ga4__funnel_steps` (pass/fail = green dbt test).
- The monotonicity unit test passes: a purchase-only session has every `reached_*` flag = true, proving `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase`.
- The sessionize golden test passes: (`u1`, `100`) two events collapse to one session, `landing_page` = the earliest-timestamp `page_location`, and a `gclid` + `cpc` session resolves to `Paid Search` via `channel_group_case()`.
- A returning user's session takes its `event_params` source/medium, **not** `traffic_source` (no first-touch leak).
- `channel_group` is one of exactly the 10 canonical groups; `session_revenue ≥ 0` and is deduped (one value per `transaction_id`).

Common mistakes

- traffic_source first-touch leak (highest-value bug).** Event-level `traffic_source.*` is GA4 **user first-touch** - identical on every session that user ever has. Using it as the session

source mis-credits every returning session to its original acquisition channel and corrupts every channel-level rate (Simpson's-paradox-grade). Fix: prefer session-scoped `event_params.source/medium` (surfaced as `event_source / event_medium` in staging) and `coalesce( ... )` to `traffic_source.*` **only when the session value is null** (DATA_MODEL §5.3).

- **Not dropping NULL `ga_session_id`**. GA4 emits session-less events (e.g. `first_open / first_visit`) with null `ga_session_id`; they cannot be sessionized. Filter `where ga_session_id is not null` at the intermediate layer - otherwise they inflate counts and break `session_key not null` (DATA_MODEL §5.1).
- **Wrong session_key expression.** The key is `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`, built **only** via the `sessionize()` macro. Never `FARM_FINGERPRINT`, never `COUNT(*)`, never a different separator - any drift silently changes distinct-count and the unit test catches it. `sessions = COUNT(DISTINCT session_key)`.
- **Occurrence-based (`did_*`) flags instead of `reached_*` max-downstream.** A session that purchased via a deep link without an explicit `add_to_cart` event would show `did_purchase=1, did_add_to_cart=0` - a step rate > 1 and a non-monotonic funnel. Use `LOGICAL_OR(event_name IN (this stage .. purchase))` so `reached_X` is true if the session fired X **or any later stage**. The `did_*` names are retired (CLAUDE.md §5).
- **Not deduping `session_revenue`**. A session with two `purchase` events (export retry, multi-stream) double-counts revenue. Take `MAX(purchase_revenue_in_usd) OVER (PARTITION BY session_key, transaction_id)` then sum across distinct transactions, never `SUM` of raw event rows (DBT_GUIDE §4.2).
- **`landing_page = any page, not the earliest`**. Use `ARRAY_AGG(page_location IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[SAFE_OFFSET(0)]` - the `page_location` of the minimum-timestamp event, not an arbitrary non-null value (DATA_MODEL §5.2).
- **Skipping golden tests because structural tests "pass".** `not_null / unique / accepted_values` catch shape, never correctness. The keystones earn the deepest test investment in the project - write the §6.5 unit tests first (DBT_GUIDE §4.4, §6.5).
- **Building before staging is green.** M3 depends on M2; a failed test on staging poisons the intermediates. Always `dbt build` (interleaves model+test in DAG order), never `dbt run` then `dbt test` separately.

M4 - Marts

Objective

Build the **consumption layer** - the only models the semantic layer (and therefore the agents) are allowed to read. Marts are **wide, denormalized** facts (each row carries its own `device_category`, `channel_group`, `country`, `is_new_user`, `event_date`) so the semantic layer slices with a single `GROUP BY` and no runtime joins, serving the ≤ 5 GiB byte budget. There are exactly **11 marts** across three folders - **core** (session/funnel spine + conformed dims), **finance** (orders + line items), **growth** (decomposition rollups + retention). Plus one snapshot (`snap_dim_items`, SCD2 on item price/category) and two singular tests. The two hardest gates: **revenue reconciles to the cent** against raw GA4, and **dim_channels holds exactly 10 channel groups** - no 11th, no "Paid Other".

Files to create

Core (`models/marts/core/`):

Path	Copy from
<code>fct_sessions.sql</code>	DBT_GUIDE.md §5.4 (grain/PK/cols); DATA_MODEL.md §5.7
<code>fct_funnel.sql</code>	DBT_GUIDE.md §5.8 (full SQL); DATA_MODEL.md §5.8 - primary session grain
<code>fct_daily_funnel.sql</code>	DBT_GUIDE.md §5.9 (full SQL); DATA_MODEL.md §5.9 - additive counts, no stored rates
<code>dim_users.sql</code>	DBT_GUIDE.md §5.4 (dim_users); DATA_MODEL.md §3
<code>dim_items.sql</code>	DBT_GUIDE.md §5.4 (dim_items); DATA_MODEL.md §3
<code>dim_channels.sql</code>	DBT_GUIDE.md §5.11 (full SQL - the 10-row literal); DATA_MODEL.md §3
<code>dim_date.sql</code>	DBT_GUIDE.md §5.4 (dim_date); DATA_MODEL.md §3
<code>core__schema.yml</code>	DBT_GUIDE.md §5.13 (core block) + §6.1

Finance (`models/marts/finance/`):

Path	Copy from
<code>fct_orders.sql</code>	DBT_GUIDE.md §5.10 (full SQL - deduped per <code>transaction_id</code>)
<code>fct_order_items.sql</code>	DBT_GUIDE.md §5.5 (fct_order_items); DATA_MODEL.md §3
<code>finance__schema.yml</code>	DBT_GUIDE.md §5.13 (finance block; <code>test_revenue_reconciles</code> on <code>gross_revenue</code>)

Growth (`models/marts/growth/`):

Path	Copy from
<code>fct_funnel_by_dim.sql</code>	DBT_GUIDE.md §5.6/§5.10 catalog; DATA_MODEL.md §5.10 (long-format, one dim at a time)
<code>fct_cohorts.sql</code>	DBT_GUIDE.md §5.12 (full SQL - fixed-denominator retention)
<code>growth__schema.yml</code>	DBT_GUIDE.md §5.13 (growth block)

Snapshot + singular tests:

Path	Copy from
<code>snapshots/snap_dim_items.sql</code>	DBT_GUIDE.md §5.4 note + project <code>snapshots:</code> config (target_schema <code>snapshots</code> , tag <code>snapshot</code>); SCD2 on item price / category
<code>tests/assert_funnel_monotonicity.sql</code>	DBT_GUIDE.md §6.3 (authored in M3; goes green here once <code>fct_funnel</code> exists)
<code>tests/assert_session_conversion_rate_bounds.sql</code>	DBT_GUIDE.md §6.3 (authored in M3; goes green here once <code>fct_daily_funnel</code> exists)

The five **base grains** the semantic layer resolves against are exactly `fct_funnel` , `fct_sessions` , `fct_orders` , `fct_order_items` , `fct_cohorts` ; the rest are conformed dims (`dim_*`) or pre-aggregated rollups (`fct_daily_funnel` , `fct_funnel_by_dim`) read indirectly. Materializations: the three session-grain core facts (`fct_sessions` , `fct_funnel` , `fct_daily_funnel`) are `incremental` (`insert_overwrite` , `partition_by` event_date , `cluster_by` device_category , `channel_group` , `require_partition_filter=true`); everything else (`dim_*` , finance, growth) is `table` .

Why marts are **wide (denormalized)** and not normalized: BigQuery is a columnar scan engine where column projection is nearly free and joins are comparatively expensive, so carrying the handful of low-cardinality dims physically on each fact costs trivial storage and removes the join from the hot path - exactly what serves the ≤5 GiB/run byte budget and <5 min time-to-diagnosis. Surrogate keys (`session_key` , `order_key` , `channel_key` , `date_key` , `user_key` , `item_key` , `order_item_key`) still exist so `relationships` tests can enforce referential integrity, but the agents never need those joins at query time. `dim_channels` , `dim_date` , and `dim_users` are **conformed** - the same physical dim is shared by every fact - so a slice by `channel_group` means the same thing everywhere, which is what lets Decompose pivot any metric across the same canonical axes (DBT_GUIDE §5.2).

Honesty on the static sample. The incremental config (3-day `is_incremental()` lookback + `insert_overwrite` partition replacement) is the **production** design - wired and correct so the moment Helios points at a live daily GA4 export it works without code changes. The public dataset is frozen (2020-11-01..2021-01-31), so no new shards land and the `is_incremental()` branch never fires; in dev always reset with `dbt build --full-refresh` (3 months rebuild in seconds, byte-identical, idempotent).

Commands to run

```
.venv\Scripts\activate

# build order is the DAG (dbt resolves it from ref()); on the static sample reset clean:
dbt build --full-refresh          # whole DAG incl. tests, idempotent on the frozen sample

# selective builds (model + all upstream) while iterating:
dbt build --select +fct_funnel    # int keystones + fct_sessions + fct_funnel
dbt build --select +fct_daily_funnel # also pulls fct_funnel underneath
dbt build --select +fct_orders    # fct_sessions (for wide dims) + fct_orders
dbt build --select marts.core marts.finance marts.growth

# snapshot (SCD2 history on item price/category):
dbt snapshot

# per-model tests (fast checks):
dbt test --select fct_orders      # incl. revenue_reconciles
dbt test --select dim_channels    # incl. accepted_values = the 10
```

Always `dbt build` (interleaves model + test in DAG order), never `dbt run` then `dbt test` - a separate run materializes the whole graph on bad upstream data and only discovers it afterward.

Tests to run

```
# 1. Revenue reconciles TO THE CENT (custom generic test) - DBT_GUIDE §6.4
dbt test --select fct_orders
#   test_revenue_reconciles on gross_revenue: |mart_total - raw_total| / raw_total ≤ tolerance.
#   fct_orders uses tolerance 0 (to the cent); fct_funnel.session_revenue uses the 0.5% default.

# 2. dim_channels = EXACTLY 10 groups - DBT_GUIDE §5.13 / §6.1
dbt test --select dim_channels
#   channel_key unique+not_null; channel_group unique+not_null; accepted_values = the 10:
#   Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social,
#   Email, Affiliates, Referral, Other. (also: SELECT count(*) FROM helios.dim_channels = 10)

# 3. PK uniqueness + referential integrity on every mart - DBT_GUIDE §5.13
dbt test --select marts
#   every grain PK unique+not_null; relationships FKs (fct_funnel.channel_group → dim_channels,
#   fct_orders.session_key → fct_sessions, fct_order_items.item_key → dim_items, ...).
```

```
# 4. Rollup monotonicity + retention bounds
dbt test --select fct_daily_funnel fct_cohorts assert_funnel_monotonicity
        assert_session_conversion_rate_bounds
#   fct_daily_funnel: purchasing_sessions ≤ sessions (expression_is_true);
#   fct_cohorts: retention_rate in [0,1] (fixed-denominator);
#   singular monotonicity returns 0 rows.

# 5. Whole-DAG green:
dbt build --full-refresh && dbt source freshness
```

Success criteria

- `dbt build` passes the whole DAG including all tests (green).
- **Revenue reconciles to the cent:** `test_revenue_reconciles` on `fct_orders.gross_revenue` passes at tolerance 0 - mart revenue equals raw `SUM(purchase_revenue_in_usd)` over `event_name='purchase'`. `fct_funnel.session_revenue` reconciles within 0.5%, and session-grain `revenue` equals order-grain `gross_revenue` at the grand total.
- `dim_channels` = **exactly 10 rows**, one per canonical group; `accepted_values` passes with no 11th group.
- Every mart's grain PK is unique + not_null; every conformed FK passes its `relationships` test.
- `fct_daily_funnel` stores **only additive counts** (no rates) and satisfies `purchasing_sessions ≤ sessions`; rates are derived later in the semantic layer as `SUM(num)/SUM(den)`.
- `fct_daily_funnel` aggregates `fct_funnel` (which carries `session_revenue`), not `int_ga4__funnel_steps` - revenue is present at the rollup grain.
- `fct_cohorts.retention_rate` is in `[0,1]` and monotonically non-increasing (fixed original-cohort denominator).

Common mistakes

- **Revenue not deduped per `transaction_id`**. GA4 emits retry/multi-stream duplicate `purchase` rows for one order. `fct_orders` must collapse to one row per `transaction_id` (e.g. `GROUP BY order_key` with `ANY_VALUE( ... )`); failing to dedup double-counts revenue and AOV and breaks `test_revenue_reconciles` (DBT_GUIDE §5.10).
- **Aggregating non-`_in_usd` columns or including shipping/tax in revenue.** Use only the `_in_usd` twins (`purchase_revenue_in_usd`); never the native-currency columns. Revenue excludes shipping and tax (those are separate columns: `shipping_value_in_usd`, `tax_value_in_usd`); `net_revenue = gross_revenue - refund_value_in_usd` (CLAUDE.md §5, DBT_GUIDE §5.10).

- **Session-grain revenue not reconciling to order-grain gross_revenue.** `fct_funnel.session_revenue` (deduped per transaction) and `fct_orders.gross_revenue` (deduped per transaction) must tie at the grand total. If they diverge, one path is double-counting or dropping untagged purchases (`ecommerce.transaction_id IS NULL` is excluded in `fct_orders`).
- **Inventing an 11th channel group.** `dim_channels` enumerates **exactly 10** rows; there is no "Paid Other". Do not duplicate channel logic anywhere - it lives only in `channel_group_case()` (`macros/channel_group.sql`); `dim_channels` just enumerates the same 10 strings as the conformed dimension (DBT_GUIDE §5.11). A stale incremental partition can drift the mart from the macro, which is why `accepted_values` is tested at the *mart* the agents read, not just the macro (DBT_GUIDE §6.1).
- **Storing rates in fct_daily_funnel.** Store **only additive counts** (`sessions`, `view_item_sessions` ... `purchasing_sessions`, `transactions`, `revenue`). Rates (`session_conversion_rate`, step rates) are derived in `semantic_layer.yaml` as `SUM(num)/SUM(den)` after grouping. Pre-dividing breaks re-aggregation across slices and reintroduces Simpson's paradox (DBT_GUIDE §5.9, DATA_MODEL §5.9).
- **Averaging per-segment ratios.** Never average pre-divided segment rates to get an aggregate. Always `SUM(numerator)/SUM(denominator)` after grouping - averaging ratios is a textbook Simpson's-paradox artifact (CLAUDE.md §5).
- **fct_daily_funnel aggregating the wrong upstream.** It must aggregate `fct_funnel` (carries `session_revenue`), not `int_ga4__funnel_steps`. Dropping revenue here breaks the eval's dollar-at-risk labels (CLAUDE.md §8, DBT_GUIDE §5.9).
- **SELECT * over events_* / no maximum_bytes_billed / scanning all 90+ shards.** Prune via `_TABLE_SUFFIX` (the sample window is `'20201101' .. '20210131'`); the incremental config's `require_partition_filter=true` makes a date-less query error rather than full-scan. Name every column explicitly.
- **Running dbt run then dbt test separately, or referencing ephemeral models in singular tests.** `dbt build` interleaves model + test in DAG order so a poisoned upstream aborts before reaching the marts. Singular tests must reference the *materialized* marts (`fct_funnel`, `fct_daily_funnel`), not the ephemeral `int_ga4__*` (which inline as CTEs and cannot be selected from in a standalone test).